



BURSA ULUDAĞ ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ

2025/2026

BAHAR YARIYILI

VERİ YAPILARI DERSİ PROJE RAPORU

İBRAHİM ALPTEKİN - 032090138

AHMET ŞEREF GÖLCÜK - 031890087

SEYMEN YİĞİT CEYLAN - 032090022

SADETTİN ŞAHİN - 032190020

1. YAZILIM MİMARİSİ VE UML DİYAGRAMLARI

Projemiz, kodun tekrar kullanılabilirliğini ve sürdürülebilirliğini sağlamak amacıyla Nesne Yönelimli Programlama prensiplerine tam uyumlu olarak tasarlanmıştır. Sistemin bileşenleri ve bu bileşenler arasındaki ilişkiler UML standartlarıyla modellenmiştir.

1.1. UML Sınıf Diyagramı Analizi

Sistemimiz temel veri yapıları, graf modeli ve algoritma motoru olmak üzere modüler sınıflara ayrılmıştır. Sınıflarımız ve içerdiği nitelikler şu şekildedir:

SINIF TANIMLAMALARI

1. Node (Struct)

PCB üzerindeki her bir elektronik bileşeni veya lehim noktasını temsil eder.

- **Nitelikler:**

- - id: int (Düğümün benzersiz kimlik numarası)
- - x: float, - y: float (Raylib arayüzündeki fiziksel ekran koordinatları)

2. Edge (Struct)

İki düğüm arasındaki fiziksel veya elektriksel bağlantıyı temsil eder.

- **Nitelikler:**

- - src: Node* (Başlangıç düğümünün referansı)
- - dest: Node* (Bitiş düğümünün referansı)
- - weight: float (İki düğüm arasındaki Öklid mesafesi veya maliyet)

3. PCBGraph (Class)

Sistemin ana veri modelidir. Tüm düğümleri ve kenarları bir koleksiyon içinde yönetir.

- **Nitelikler:**

- - vertices: std::vector<Node>
- - edges: std::vector<Edge>

- **Metotlar:**

- + addVertex(x: float, y: float): void
- + addEdge(srcID: int, destID: int): void
- + getEdges(): std::vector<Edge>
- + getVertices(): const std::vector<Node>&

4. UnionFind (Class)

Kruskal algoritmasının döngü kontrolünü $O(1)$ 'e yakın bir sürede yapmasını sağlayan optimize veri yapısıdır.

- **Nitelikler:**
 - - parent: std::vector<int>
 - - rank: std::vector<int>
- **Metotlar:**
 - + UnionFind(n: int)
 - + find(i: int): int
 - + unite(x: int, y: int): bool

5. MSTOptimizer (Class)

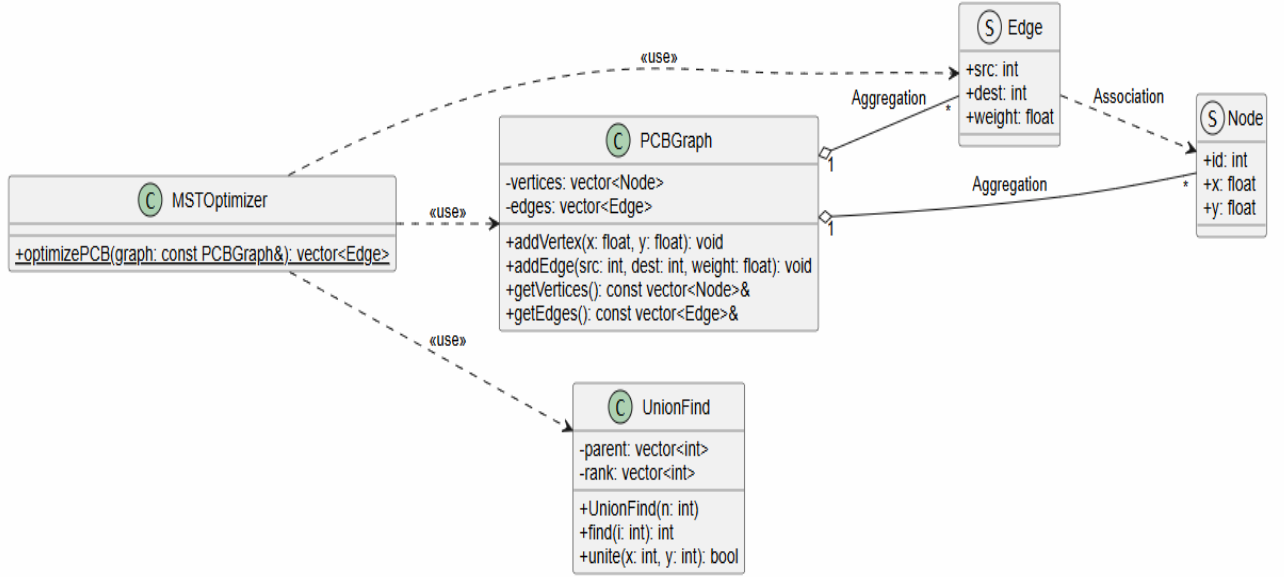
Matematiksel işlemlerin yapıldığı izole edilmiş statik sınıftır.

- **Metotlar:**
 - + static solveKruskal(graph: const PCBGraph&): std::vector<Edge>

SINIFLAR ARASI İLİŞKİLER

- **Aggregation (Birleştirme):** Graph sınıfı mantıksal olarak Node ve Edge nesnelerini kapsar. Uygulama döngüsü boyunca graf nesnesi bu yapıları içinde barındırır.
- **Composition (Oluşturma):** Edge (Kenar) sınıfı tamamen Node (Düğüm) sınıfına bağımlıdır. Uçlarında iki adet lehim noktası (düğüm) olmayan bir bakır yol (kenar) var olamaz.
- **Dependency (Bağımlılık):** Algorithms sınıfı görevini yerine getirebilmek için dışarıdan Graph ve DSU yapılarına ihtiyaç duyar.

SINIF DİYAGRAMI

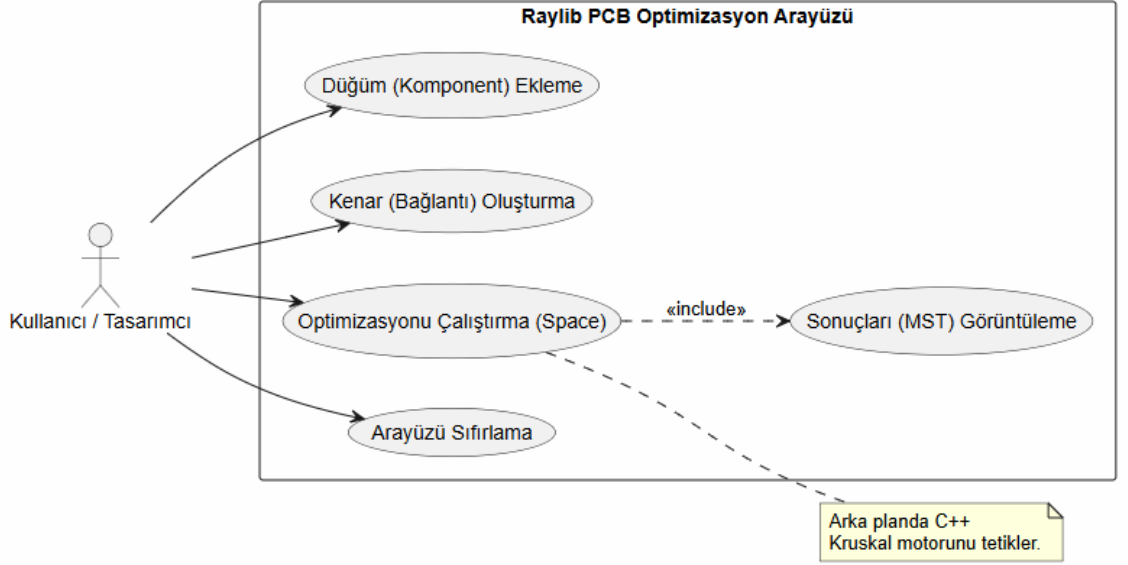


1.1. UML USE CASE DİYAGRAMI ANALİZİ

Sistemin kullanıcısı ile arka plandaki fonksiyonların etkileşimini modelleyen diyagramdır.

- **Aktör:** Kullanıcı
- **Kullanım Senaryoları:**
 1. **Düğüm Ekleme:** Kullanıcının Raylib ekranına farenin sol tuşu ile tıklayarak karta yeni component/nokta eklemesi.
 2. **Kenar Oluşturma:** Sistem, eklenen her yeni düğümü mevcut düğümlerle otomatik veya manuel olarak ilişkilendirir.
 3. **Optimizasyonu Çalıştırma:** Kullanıcının "Space" (Boşluk) tuşuna basarak C++ algoritma motorunu tetiklemesi.
 4. **Sonuçları Görmü:** Optimizasyon tamamlandığında, hesaplanan en kısa yolların arayüzde yeşil ve kalın çizgilerle gösterilmesi.
 5. **Arayüzü Sıfırlama:** Tüm graf verilerinin bellekten silinip ekranın temizlenmesi.

USE CASE DİYAGRAMI



2. VERİ YAPILARI VE ZAMAN KARMAŞIKLIĞI ANALİZİ

Bu projede donanım üzerindeki fiziksel yolları (PCB hatlarını) minimum maliyetle birleştirmek amacıyla Kruskal Algoritması kullanılmıştır. Çok sayıda elektronik komponentin (düğümün) bulunduğu senaryolarda işlemci darboğazlarını önlemek ve Raylib arayüzünün 60 FPS hızında donmadan çalışmasını sağlamak için veri yapıları özel olarak optimize edilmiştir.

2.1. KULLANILAN VERİ YAPILARI VE OPTİMİZASYONLAR

2.1.1. Disjoint Set Union

Kruskal algoritmasının en büyük maliyeti, graf üzerinde döngü oluşup oluşmadığını kontrol etmektir. Standart bir DFS (Derinlik Öncelikli Arama) kullanılsaydı bu işlem $O(V)$ zaman alacaktı. Projemizde bu maliyeti teorik olarak $O(1)$ seviyesine indirmek için özel bir DSU veri yapısı kodlanmıştır.

DSU yapısına uygulanan kritik optimizasyonlar şunlardır:

- **Path Compression (Yol Sıkıştırma):** find() (temsilci bulma) metodu çalıştığında, aranan düğüm doğrudan kümenin kök düğümüne (root) bağlanır. Böylece bir sonraki arama işleminin ağaçta derinlere inmesi engellenerek işlem süresi kısaltılır.
- **Union by Rank (Dereceye Göre Birleştirme):** İki küme birleştirilirken (unionSet metodu), her zaman derinliği (rank) küçük olan ağaç, derinliği büyük olan ağacın altına eklenir. Bu sayede ağacın dengesiz bir şekilde uzaması (Linked List formuna dönüşmesi) fiziksel olarak engellenir.

Bu iki optimizasyon sayesinde DSU üzerindeki her bir işlemin zaman karmaşıklığı Ters Ackermann Fonksiyonu olan $\alpha(V)$ 'ye düşürülmüştür. $\alpha(V)$ pratikte 4'ten küçük bir sabittir ve bu da işlemi $O(1)$ (Sabit Zaman) hızına yaklaştırır.

2.2. Zaman Karmaşıklığı Analizi

Projenin temelini oluşturan Kruskal algoritmasının uçtan uca zaman karmaşıklığı $O(E \log E)$ olarak hesaplanmıştır (Burada V: Düğüm/Komponent sayısı, E: Kenar/Bağlantı sayısıdır).

Bu sonuca algoritmamızın iki ana aşaması analiz edilerek ulaşılmıştır:

1. Kenarların Sıralanması

Algoritma başlamadan önce tüm olası yollar (kenarlar), maliyetlerine göre küçükten büyüğe sıralanmaktadır. Bu işlem için C++ standart kütüphanesindeki İçer Dönük Sıralama algoritması kullanılmıştır.

- **İşlem Maliyeti:** $O(E \log E)$

2. Döngü Kontrolü ve Birleştirme

Sıralanan tüm kenarlar için (toplam E adet) sırayla döngü kontrolü yapılır. Her bir kenar için yapılan DSU find ve union işlemleri $\alpha(V)$ zaman almaktadır.

- **İşlem Maliyeti:** $O(E \times \alpha(V))$

Nihai Sonuç:

Matematiksel asimptotik analizde, sıralama işleminin maliyeti ($E \log E$), birleştirme işleminin maliyetinden ($E \times \alpha(V)$) daha büyük olduğu için sistemi domine eder. Bu nedenle Kruskal algoritmasının projemizdeki genel zaman karmaşıklığı $O(E \log E)$ olarak kesinleşmiştir.

2.3. Alan Karmaşıklığı Analizi

Grafın RAM (Bellek) üzerinde ne kadar yer kaplayacağını analizi şu şekildedir:

- Tüm düğümlerin koordinatları ve ID'leri bir `std::vector` içinde saklanır: $O(V)$ alan
- Tüm olası bağlantılar (edges) başka bir `std::vector` içinde saklanır: $O(E)$ alan
- DSU yapısı, ebeveyn (parent) ve derece (rank) dizileri için $2 \times V$ boyutunda bellek tahsis eder: $O(V)$ alan

Bu değerler toplandığında projenin genel alan (bellek) karmaşıklığı $O(V+E)$ olarak ölçülür. Bu doğrusal bellek kullanımı, sistemin kısıtlı kaynaklara sahip cihazlarda bile sızıntı yapmadan stabil çalışmasını sağlamaktadır.